

DESIGN PATTERNS JUSTIFICATION & CLASS DIAGRAMS

E-commerce Platform



*Arjola Metaliaj, Ederta Kariqi, Asja Xhelezi, Stilianos Troka, Elda
Toska, Sindi Terihati, Elisa Spaho*

STATE PATTERN

What is the State Pattern?

The State Pattern is a behavioral design pattern that allows an object to change its behavior depending on its current state. Instead of using large conditional statements to manage different behaviors, the pattern separates each state into its own class.

The main object, called the context, stores a reference to a state object and delegates state-dependent behavior to it. As the state changes, the object behaves differently without modifying its own core structure.

This pattern improves maintainability, scalability, and readability by encapsulating state-specific logic into separate classes.

RELEVANCE TO OUR SYSTEM DESIGN

The State Pattern was selected because the order processing system naturally operates through a sequence of lifecycle states.

An order does not behave the same throughout its existence. Different operations become valid or invalid depending on the current order status.

For example:

- A pending order can be confirmed or cancelled.
- A confirmed order can be shipped.
- A shipped order can be delivered.
- A delivered order can only be refunded.

Because of this changing behavior, the State Pattern is highly suitable for modeling the order lifecycle.

Instead of implementing complex conditional statements such as:

```
if status == "Pending"  
if status == "Confirmed"  
if status == "Shipped"
```

The system delegates behavior to specialized state classes making for a more elegant and extensible architecture

.

DESIGN CHOICES MADE

1. Order as the Context Class

The Order class was designed as the context object because it is the entity whose behavior changes during the order lifecycle.

The class stores:

- order information,
- the current state object,
- and the high-level operations available to the system.

The Order object delegates state-dependent operations to the active OrderState.

2. Abstract OrderState Superclass

An abstract superclass named OrderState was used instead of an interface.

This decision was made because:

- the lecture slides model the pattern using inheritance,
- all states share common structure and behavior,
- and each state requires access to the owning order object.

The superclass also contains:

- a reference to the Order,
- shared operation definitions,
- and a constructor used by all concrete states.

This creates stronger consistency between states.

3. Concrete State Classes

Separate classes were created for:

- PendingState
- ConfirmedState
- ShippedState
- DeliveredState

Each class contains only the operations valid for that specific state.

For example:

- PendingState allows confirmation and cancellation.
- ConfirmedState allows shipping.
- DeliveredState allows refunds only.

This design prevents invalid operations and models real business rules more accurately.

DESIGN CHOICES MADE

4. Aggregation Relationship

An aggregation relationship was used between Order and OrderState. This was chosen because:

- the order maintains a reference to a current state object,
- but the state logic remains separated from the order itself,
- supporting loose coupling and delegated behavior.

5. Lifecycle-Focused Operations

The operations included in the diagram were intentionally focused on lifecycle behavior:

- `confirmOrder()`
- `cancelOrder()`
- `shipOrder()`
- `deliverOrder()`
- `refundOrder()`

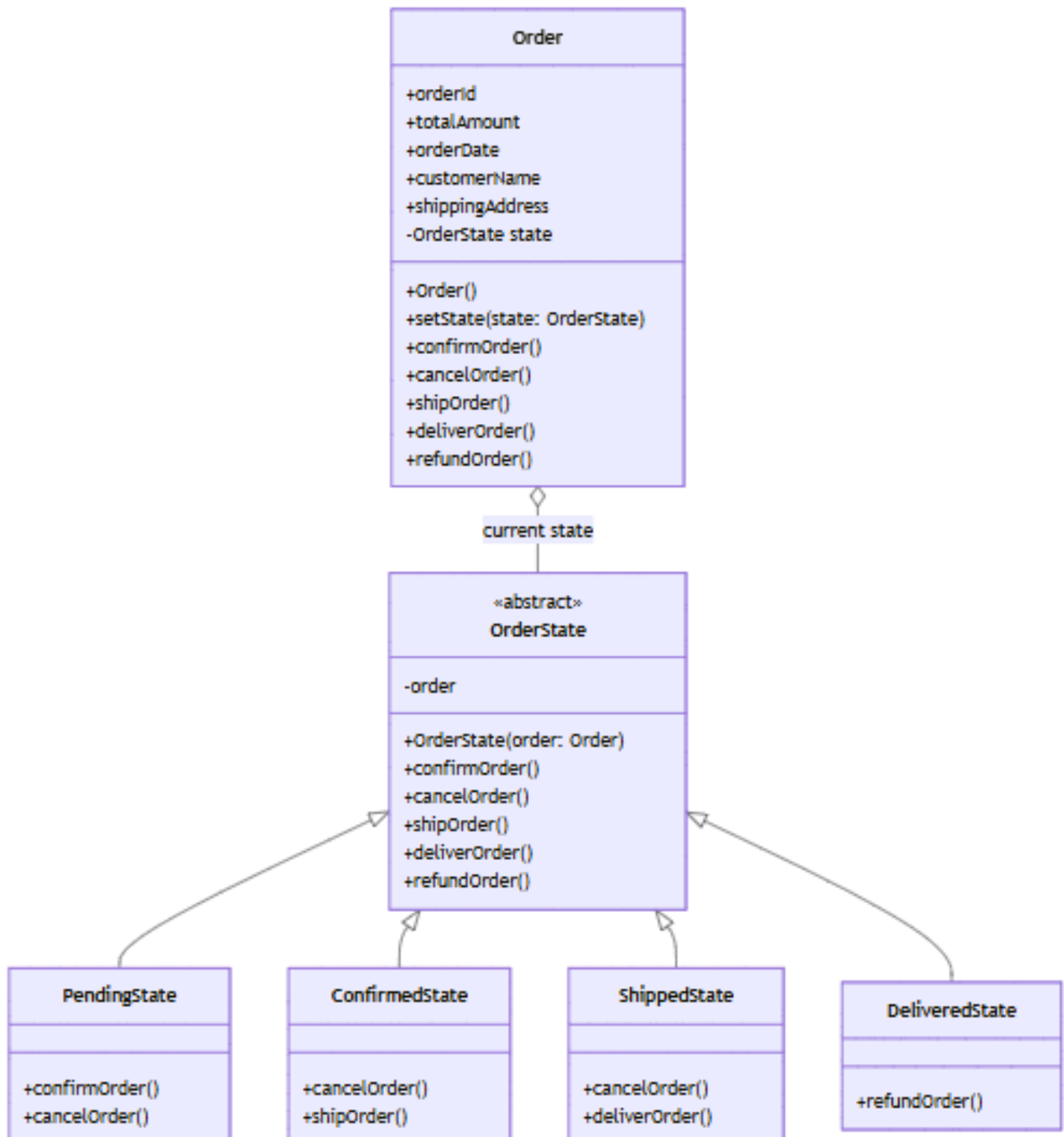
These operations directly represent transitions and actions affected by state changes, which aligns closely with the purpose of the State Pattern.

BENEFITS OF THIS DESIGN

Using the State Pattern provides several advantages in the system:

- Eliminates large conditional statements.
- Encapsulates state-specific behavior.
- Makes the order lifecycle easier to maintain.
- Simplifies adding new states in the future.
- Improves readability and scalability.
- Models real-world order processing behavior more accurately.

CLASS DIAGRAM



FAÇADE PATTERN

What is the Façade Pattern?

The Façade Pattern is a structural design pattern that provides a simplified interface to a complex subsystem made up of many classes and operations.

Instead of the client interacting directly with multiple services, the client communicates only with the façade, which internally coordinates the workflow between subsystem components.

The pattern helps reduce complexity, hide implementation details, and make the system easier to use and maintain.

RELEVANCE TO OUR SYSTEM DESIGN

The Façade Pattern was selected because the checkout process in the e-commerce platform naturally involves multiple independent services that must work together in a specific sequence.

During checkout:

- payment must be processed,
- stock must be validated and reserved,
- shipment information must be created,
- invoices must be generated,
- and confirmation notifications must be sent.

Without a façade, the client would need to communicate with every service individually, creating tightly coupled and difficult-to-maintain code.

Because of this complex interaction between services, the Façade Pattern is highly suitable for simplifying the checkout workflow.

Instead of implementing complicated coordination logic inside the frontend or controller classes, the system delegates the orchestration responsibility to a dedicated façade class.

DESIGN CHOICES MADE

1. CheckoutPage as the Client

The CheckoutPage class was designed as the client because it represents the component initiating the checkout process.

The page does not directly communicate with all subsystem services. Instead, it interacts only with:

- ICheckoutFacade

through the operation:

- checkout(order : Order)

This keeps the frontend simpler and reduces subsystem dependency exposure.

2. ICheckoutFacade Interface

An interface named ICheckoutFacade was used to define the checkout operation.

This decision was made because:

- it improves abstraction,
- reduces coupling,
- and allows flexibility for future implementations.

The client depends on the interface rather than the concrete façade implementation.

3. CheckoutFacade as the Central Coordinator

CheckoutFacade was designed as the main façade class because it coordinates the entire checkout workflow.

The class internally manages:

- payment processing,
- stock validation and reservation,
- shipment creation,
- invoice generation,
- and notification delivery.

The façade hides the complexity of subsystem communication and exposes only a single simplified operation to the client.

DESIGN CHOICES MADE

4. Specialized Service Classes

Separate subsystem classes were created for:

- PaymentService
- InventoryService
- ShippingService
- InvoiceService
- NotificationService

Each class contains only the operations related to its specific responsibility.

For example:

- PaymentService handles payment processing.
- InventoryService validates and reserves stock.
- ShippingService creates shipment information.

This design improves modularity and separates responsibilities clearly.

BENEFITS OF THIS DESIGN

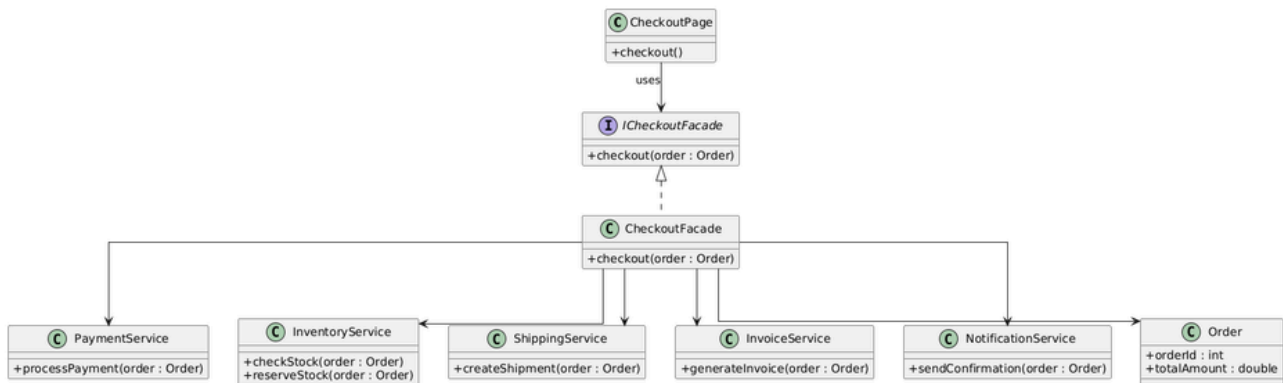
Using the Façade Pattern provides several advantages in the system:

- Reduces complexity exposed to the client.
- Simplifies the checkout workflow.
- Centralizes orchestration logic.
- Improves maintainability and readability.
- Reduces coupling between components.
- Makes future extensions easier.
- Hides subsystem implementation details from the client.

One possible drawback is that the façade could eventually become too large and overloaded with responsibilities if too much functionality is added to it.

However, when kept focused on workflow coordination, the Façade Pattern creates a cleaner and more scalable architecture for the checkout system.

CLASS DIAGRAM



CLASS DIAGRAM WITH ATTRIBUTES

